Addition of two numbers

R Vijayshri

July 2026

Introduction

One of the most fundamental operations we learnt after learning numbers is addition. We use addition (decimal) in our everyday life. Why decimal? Perhaps because we have 10 fingers and also the number 10 has convenient properties. However, addition can also be done among other bases and in the context of computers, addition is done between two $n-$ bit numbers. All the other binary arithmetic operations (like multiplication, subtraction) can be done using the addition circuit. Arithmetic operations are required to execute many important algorithms and processes.

Binary Addition Explained

1-bit Addition

Just like in decimal addition (base 10), when we add two single digit numbers and the sum is greater than 9, we get a carry, so in binary addition (base 2) when the sum of two bits is greater than 2, we get a carry.

For single-bit numbers $a, b \in \{0, 1\}$, the rules are:

a	b	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

We notice that the sum function is essentially the XOR gate function and the Carry function is nothing but the multiplication gate function.

$$\text{Sum} = a \oplus b, \quad \text{Carry} = a \cdot b$$

2-bit Addition

Now, let us move onto addition of 2 bit binary numbers. Just as we learnt to add with the carry in decimal numbers, so shall we do here.

a	b	c_{in}	Sum	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Here, c_{in} is the carry we add and c_{out} is the carry we carry forward.

For example, let us say that we want to add 11 with 01.

$$\begin{array}{cccc} & \mathbf{1} & \mathbf{1} & \\ & & & \\ & 1 & 1 & \\ + & 0 & 1 & \\ \hline 1 & 0 & 0 & \end{array}$$

For two-bit numbers $A = a_1a_0$ and $B = b_1b_0$:

$$\begin{aligned} s_0 &= a_0 \oplus b_0 \\ c_0 &= a_0 \cdot b_0 \\ s_1 &= a_1 \oplus b_1 \oplus c_0 \\ c_1 &= (a_1 \cdot b_1) + (c_0 \cdot (a_1 \oplus b_1)) \end{aligned}$$

So the result is:

$$S = s_1s_0, \quad \text{Carry out} = c_1$$

In the example above, $S = 00$, Carry out $= 1$.

This can be represented through the following circuit -

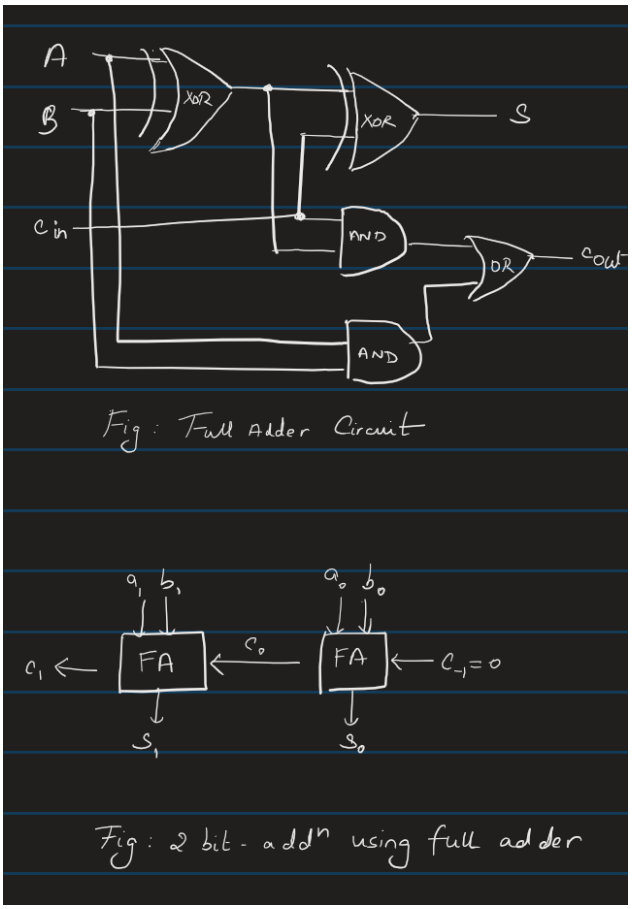


Figure 1: Full adder circuit and its use in 2-bit addition

n-bit Addition

For n-bit numbers addition, $A = a_{n-1} \dots a_0$ and $B = b_{n-1} \dots b_0$, the idea is similar to the addition of two bit numbers, except we continue the process till the nth bit.

$$
\begin{aligned}
s_i &= a_i \oplus b_i \oplus c_{i-1}, \\
c_i &= (a_i \cdot b_i) + (c_{i-1} \cdot (a_i \oplus b_i)), \quad i = 0, 1, \dots, n-1
\end{aligned}
$$

with initial carry $c_{-1} = 0$. The final sum is $S = s_{n-1} \dots s_0$ and the last carry c_{n-1} may extend the result to $(n+1)$ bits.

So far, we assumed that the two numbers we are adding are unsigned. However, we can still implement this addition with signed numbers. The way we go about this is, if A and B are two unsigned integers in the range 0 to $2^n - 1$, then $(-A)$ is defined as $2^n - A$ and respectively for B. This is referred to as the 2's complement of A, as $2^n - A$ is essentially complementing every bit of A with an extra bit 1 as the nth bit.

So, to do $(-A) + B$, we just do $(2^n - A) + B$.

Similarly, $A + (-B) = A + 2^n - B$

And, $(-A) + (-B) = (2^n - A) + (2^n - B) = 2^n - (A + B)$

We can modify the earlier circuit so that we can express both $A + B$ and $A - B$.

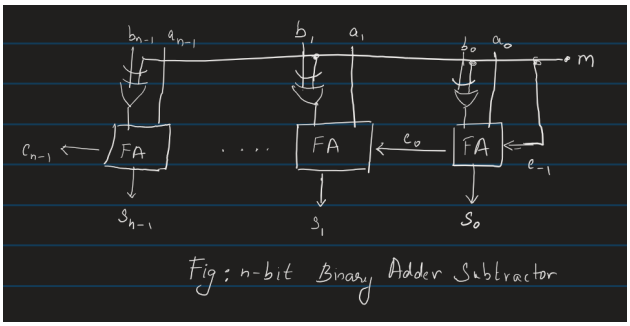


Figure 2: $n-$bit binary adder-subtractor

Here, M acts as a toggle, i.e. when $M = 0$, the result is $A + B$, and when $M = 1$, essentially, each bit of B is complemented, using the XOR function. This gives the 2's complement of B as input and so the result is $A - B$.

Conclusion

We have seen how the addition of $n-$bit numbers is done using the full adder circuit and its analogue to the decimal addition.

References

- `https://www.tutorialspoint.com/digital-electronics/digital-electronics-binary-adder-subtractor.htm`